

Docker Complete Course & Interview Guide

From Fundamentals to Advanced Concepts, with 45+ Interview Q&A

Prepared for Vikas Kumar

July 2026

Contents

1	Part 1: Docker Full Course	3
1.1	1. Introduction to Docker	3
1.1.1	1.1 What is Docker?	3
1.1.2	1.2 Why Docker? The Problem It Solves	3
1.1.3	1.3 Containers vs Virtual Machines	3
1.1.4	1.4 Docker Architecture	4
1.1.5	1.5 Installing Docker	4
1.2	2. Docker Images	4
1.2.1	2.1 What is an Image?	4
1.2.2	2.2 Common Image Commands	4
1.2.3	2.3 Writing a Dockerfile	5
1.2.4	2.4 Key Dockerfile Instructions	5
1.2.5	2.5 Image Layers and Caching	6
1.2.6	2.6 Multi-Stage Builds	6
1.3	3. Docker Containers	7
1.3.1	3.1 Container Lifecycle Commands	7
1.3.2	3.2 Common docker run Flags	7
1.3.3	3.3 Container States	7
1.4	4. Docker Networking	7
1.4.1	4.1 Network Drivers	7
1.4.2	4.2 Common Commands	8
1.5	5. Docker Storage & Volumes	8
1.6	6. Docker Compose	8
1.7	7. Docker Registries	9
1.8	8. Docker Security Best Practices	10
1.9	9. Docker Swarm (Native Orchestration)	10
1.10	10. Optimizing Docker Images	10
1.11	11. Docker in CI/CD	11
2	Part 2: Most Asked Docker Interview Questions	12
2.1	Beginner Level	12
2.2	Intermediate Level	13
2.3	Advanced Level	14

2.4	Scenario-Based Questions	16
2.5	Quick Reference: Most Common Docker Commands	17

1 Part 1: Docker Full Course

1.1 1. Introduction to Docker

1.1.1 1.1 What is Docker?

Docker is an open-source platform that packages an application and everything it needs (code, runtime, system tools, libraries, settings) into a single unit called a **container**. Containers run consistently on any machine — a developer’s laptop, a test server, or a production cloud instance — because the container carries its own environment with it.

1.1.2 1.2 Why Docker? The Problem It Solves

Before containers, teams struggled with the classic complaint: *“It works on my machine.”* Differences in OS versions, installed libraries, and configuration between development, testing, and production environments caused bugs that were hard to reproduce.

Docker solves this by:

- **Packaging dependencies with the app** — no more “missing library” surprises.
- **Isolating applications** from each other and from the host system.
- **Enabling consistent environments** across dev, staging, and production.
- **Making deployment fast and repeatable** using images as blueprints.
- **Improving resource efficiency** compared to full virtual machines.

1.1.3 1.3 Containers vs Virtual Machines

Aspect	Virtual Machines	Containers
Isolation level	Full OS per VM (hypervisor)	Process-level (shares host kernel)
Boot time	Minutes	Seconds (often milliseconds)
Size	GBs (full OS image)	MBs (just app + dependencies)
Performance	Overhead from hypervisor	Near-native performance
Density	Fewer per host	Many more per host
Use case	Running multiple full OSes	Running many lightweight, isolated apps

A VM virtualizes hardware and runs a full guest OS on top of a hypervisor. A container virtualizes the operating system — it shares the host’s kernel but has its own isolated filesystem, process tree, and network stack, achieved through Linux kernel features called **namespaces** and **cgroups**.

1.1.4 1.4 Docker Architecture

Docker uses a client-server architecture with these core components:

- **Docker Client (CLI)** — the docker command you type; sends instructions to the daemon via a REST API.
- **Docker Daemon (dockerd)** — runs on the host machine, builds/runs/manages containers, images, networks, and volumes.
- **Docker Images** — read-only templates used to create containers (built from a Dockerfile).
- **Docker Containers** — runnable instances of images.
- **Docker Registry** — stores and distributes images (e.g., Docker Hub, Amazon ECR, private registries).
- **containerd / runc** — the lower-level container runtime that actually creates and runs containers according to the OCI (Open Container Initiative) spec.

Flow: `docker build` → creates an **image** → `docker run` → daemon pulls image (if needed) and asks containerd/runc to start a **container** from it.

1.1.5 1.5 Installing Docker

On Linux (Ubuntu example):

```
sudo apt-get update
sudo apt-get install -y docker.io
sudo systemctl enable docker --now
sudo usermod -aG docker $USER    # run docker without sudo
```

On Windows/Mac: install **Docker Desktop**, which bundles a lightweight Linux VM to run the daemon.

Verify installation:

```
docker --version
docker run hello-world
```

1.2 2. Docker Images

1.2.1 2.1 What is an Image?

An image is a **read-only, layered filesystem** plus metadata (entrypoint, environment variables, exposed ports, etc.) that Docker uses to create containers. Images are built in layers — each instruction in a Dockerfile typically creates a new layer, and layers are cached and reused to speed up builds.

1.2.2 2.2 Common Image Commands

```
docker pull nginx:latest    # download an image
docker images               # list local images
```

```

docker rmi nginx:latest      # remove an image
docker image prune           # remove unused/dangling images
docker history nginx:latest  # view image layers
docker inspect nginx:latest  # view detailed metadata
docker tag nginx:latest myrepo/nginx:v1 # tag an image for a registry
docker push myrepo/nginx:v1  # push image to a registry

```

1.2.3 2.3 Writing a Dockerfile

A Dockerfile is a text file with step-by-step instructions to build an image.

```

# Use an official lightweight base image
FROM node:20-alpine

# Set working directory inside the container
WORKDIR /app

# Copy dependency manifests first (better layer caching)
COPY package*.json ./
RUN npm install --production

# Copy the rest of the application code
COPY . .

# Document the port the app listens on
EXPOSE 3000

# Set an environment variable
ENV NODE_ENV=production

# Command to run when container starts
CMD ["node", "server.js"]

```

Build and run:

```

docker build -t my-node-app:1.0 .
docker run -p 3000:3000 my-node-app:1.0

```

1.2.4 2.4 Key Dockerfile Instructions

Instruction	Purpose
FROM	Sets the base image
WORKDIR	Sets working directory for subsequent instructions
COPY / ADD	Copies files into the image (ADD also handles URL fetch/tar extraction)

Instruction	Purpose
RUN	Executes a command at build time, creates a new layer
CMD	Default command executed when the container starts (can be overridden)
ENTRYPOINT	Fixed executable for the container; CMD becomes its default arguments
EXPOSE	Documents which port the container listens on
ENV	Sets environment variables
ARG	Defines build-time variables
VOLUME	Declares a mount point for persistent/shared data
USER	Sets the user to run subsequent instructions/container as
LABEL	Adds metadata key-value pairs

1.2.5 2.5 Image Layers and Caching

Each instruction (RUN, COPY, ADD) creates a new, cached layer. If a layer hasn't changed since the last build, Docker reuses the cache instead of rebuilding it — this is why you should **order instructions from least to most frequently changing** (e.g., install dependencies before copying source code).

1.2.6 2.6 Multi-Stage Builds

Multi-stage builds keep final images small by separating the “build” environment from the “runtime” environment.

```
# Stage 1: build
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp .

# Stage 2: minimal runtime image
FROM alpine:3.19
WORKDIR /app
COPY --from=builder /app/myapp .
CMD ["/myapp"]
```

This produces a final image containing only the compiled binary and Alpine base — not the entire Go toolchain — dramatically reducing image size and attack surface.

1.3 3. Docker Containers

1.3.1 3.1 Container Lifecycle Commands

```
docker run -d --name web -p 8080:80 nginx      # create + start (detached)
docker ps                                       # list running containers
docker ps -a                                   # list all containers (incl. stopped)
docker stop web                                # gracefully stop
docker start web                               # start a stopped container
docker restart web                             # restart
docker rm web                                  # remove a stopped container
docker exec -it web bash                       # open interactive shell inside container
docker logs -f web                             # stream container logs
docker top web                                 # view running processes in container
docker stats                                   # live resource usage of containers
docker inspect web                             # detailed JSON config/state
docker cp file.txt web:/app/                   # copy file into container
```

1.3.2 3.2 Common docker run Flags

Flag	Meaning
-d	Run in detached (background) mode
-it	Interactive terminal (TTY + STDIN)
--name	Assign a custom container name
-p host:container	Publish/map a port
-v host:container	Mount a volume/bind mount
-e KEY=VALUE	Set an environment variable
--rm	Automatically remove container when it exits
--network	Attach to a specific network
--restart	Restart policy (no, on-failure, always, unless-stopped)
--memory, --cpus	Limit resource usage

1.3.3 3.3 Container States

A container moves through states: **Created** → **Running** → **Paused** → **Stopped/Exited** → **Removed**. Understanding this helps with debugging (docker ps -a to see exit codes, docker logs to see why it crashed).

1.4 4. Docker Networking

1.4.1 4.1 Network Drivers

- **bridge** (default) — private internal network on the host; containers on the same bridge can reach each other by name.

- **host** — container shares the host's network stack directly (no isolation, best performance).
- **none** — no networking.
- **overlay** — connects containers across multiple Docker hosts (used in Swarm/multi-host setups).
- **macvlan** — assigns a MAC address so the container appears as a physical device on the network.

1.4.2 4.2 Common Commands

```
docker network ls
docker network create my-network
docker network inspect my-network
docker run -d --network my-network --name appl myimage
docker network connect my-network some-container
```

Containers on a **user-defined bridge network** can resolve each other by container name via Docker's built-in DNS — this is the recommended way to let containers talk to each other (instead of the legacy default bridge, which requires `--link`).

1.5 5. Docker Storage & Volumes

Containers are ephemeral by default — data written inside a container's writable layer disappears when the container is removed. Docker offers three ways to persist data:

- **Volumes** (recommended) — managed by Docker, stored under `/var/lib/docker/volumes/`, independent of the container lifecycle, easy to back up/migrate.
- **Bind mounts** — map a specific host directory/file into the container; useful for local development (live code reload).
- **tmpfs mounts** — stored in host memory only, never written to disk; used for sensitive/temporary data.

```
docker volume create mydata
docker run -d -v mydata:/var/lib/mysql mysql
docker run -d -v $(pwd)/src:/app/src myimage      # bind mount
docker volume ls
docker volume inspect mydata
docker volume rm mydata
docker volume prune
```

1.6 6. Docker Compose

Docker Compose lets you define and run **multi-container applications** using a single YAML file.


```

version: "3.9"
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - FLASK_ENV=development
    depends_on:
      - db
    volumes:
      - ./app:/app

  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:

```

Common commands:

```

docker compose up -d      # build/create/start all services
docker compose down       # stop and remove containers, network
docker compose ps         # list services
docker compose logs -f web # tail logs for a service
docker compose exec web bash # shell into a running service
docker compose build      # rebuild images

```

1.7 7. Docker Registries

A registry stores and distributes images. **Docker Hub** is the default public registry; organizations often run private registries (Amazon ECR, Google Artifact Registry, Azure ACR, Harbor, or a self-hosted registry image) for proprietary images.

```

docker login
docker tag myapp:1.0 myuser/myapp:1.0
docker push myuser/myapp:1.0
docker pull myuser/myapp:1.0

```

1.8 8. Docker Security Best Practices

- Use minimal base images (alpine, distroless) to reduce attack surface.
- Run containers as a **non-root user** (USER instruction).
- Don't store secrets in images or Dockerfiles — use secret managers, environment injection, or Docker/Compose secrets.
- Regularly scan images for vulnerabilities (docker scout, Trivy, Snyk).
- Keep base images and dependencies updated.
- Use .dockerignore to avoid copying sensitive/unnecessary files into the build context.
- Set resource limits (--memory, --cpus) to prevent a single container from starving the host.
- Sign and verify images (Docker Content Trust) in production pipelines.
- Avoid --privileged mode unless absolutely required.

Example .dockerignore:

```
.git
node_modules
*.log
.env
Dockerfile
```

1.9 9. Docker Swarm (Native Orchestration)

Docker Swarm turns a group of Docker hosts into a single virtual cluster.

```
docker swarm init                                # initialize a swarm on this node
docker swarm join --token <token> <manager-ip>:2377 # join a worker/manager
docker service create --name web --replicas 3 -p 80:80 nginx
docker service scale web=5
docker service ls
docker stack deploy -c docker-compose.yml mystack
```

Key concepts: **Manager nodes** (maintain cluster state, schedule tasks) vs **Worker nodes** (run tasks); **Services** (desired-state definitions); **Tasks** (individual container instances of a service); built-in **load balancing** and **overlay networking**.

Swarm is Docker's native option; **Kubernetes** is the more widely adopted orchestrator in industry for complex, large-scale deployments.

1.10 10. Optimizing Docker Images

- Use multi-stage builds to strip build tools from the final image.
- Choose small base images (alpine, slim variants, or distroless).
- Combine related RUN commands with && to reduce layers.

- Clean up package manager caches in the same RUN layer (`apt-get clean && rm -rf /var/lib/apt/lists/*`).
 - Order Dockerfile instructions so rarely-changing steps (installing dependencies) happen before frequently-changing steps (copying source code), to maximize cache reuse.
 - Use `.dockerignore` to keep the build context small and fast.
-

1.11 11. Docker in CI/CD

A typical pipeline: code push → CI builds a Docker image → runs automated tests inside a container → pushes the image to a registry (tagged with commit SHA or version) → CD pipeline deploys the image to staging/production (via Compose, Swarm, Kubernetes, or a cloud service like ECS/AKS/GKE).

Benefits: identical artifacts move through every environment, rollbacks are just redeploying a previous image tag, and builds are reproducible.

2 Part 2: Most Asked Docker Interview Questions

2.1 Beginner Level

Q1. What is Docker, and why is it used? Docker is a platform for building, shipping, and running applications inside lightweight, portable containers that bundle code with its dependencies. It's used to eliminate "works on my machine" issues, speed up deployment, and improve consistency across environments.

Q2. What is the difference between an image and a container? An **image** is a static, read-only template (blueprint) built from a Dockerfile. A **container** is a running (or stopped) instance of that image, with its own writable layer, that actually executes the application.

Q3. What is a Dockerfile? A plain-text file containing a sequence of instructions (FROM, RUN, COPY, CMD, etc.) that Docker reads to automatically build an image.

Q4. What is the difference between CMD and ENTRYPOINT? CMD provides default arguments/command for the container, and is easily overridden by arguments passed to `docker run`. ENTRYPOINT sets a fixed executable that always runs; any CMD or run-time arguments are passed to it as parameters. They're often combined: ENTRYPOINT fixes the binary, CMD supplies default flags.

Q5. What is the difference between COPY and ADD? Both copy files from the build context into the image. ADD has extra features: it can automatically extract local tar archives and fetch files from remote URLs. Best practice is to prefer COPY for clarity and predictability, and only use ADD when you specifically need its extra behavior.

Q6. What is a Docker volume, and why use one instead of a bind mount? A volume is Docker-managed persistent storage, decoupled from the container's life-cycle, stored in Docker's own managed location. Volumes are portable across hosts (with the right driver), easier to back up, and don't depend on the host's directory structure — unlike bind mounts, which tie a container to a specific host path.

Q7. How do you reduce the size of a Docker image? Use a minimal base image, multi-stage builds, combine RUN layers, remove unnecessary files/cache in the same layer they were created, and use `.dockerignore` to avoid copying unneeded files.

Q8. What is .dockerignore used for? It tells Docker which files/directories to exclude from the build context, similar to `.gitignore`. This speeds up builds and prevents accidentally copying secrets or large unneeded files into the image.

Q9. How do you persist data generated by a container? By using volumes or bind mounts, mounted at container start time with `-v` or `--mount`, so the data lives outside the container's writable layer and survives container removal.

Q10. What happens to data when a container is removed? Any data written to the container's own writable layer is lost. Data in a mounted volume or bind mount is not affected, since it lives outside the container.

2.2 Intermediate Level

Q11. Explain the Docker architecture (client-server model). The Docker CLI (client) sends commands over a REST API to the Docker daemon (dockerd), which manages images, containers, networks, and volumes. The daemon delegates actual container creation/execution to containerd and runc, which follow the OCI runtime spec.

Q12. What are Docker namespaces and cgroups? Namespaces provide isolation — each container gets its own view of process IDs, network interfaces, mount points, hostname, and users, so it can't see other containers' resources. **cgroups (control groups)** limit and account for resource usage (CPU, memory, I/O) per container, preventing one container from starving others.

Q13. How does Docker networking work by default? By default, Docker creates a **bridge** network; containers get a private IP on that bridge and can reach the outside world via NAT through the host. Containers on the same **user-defined** bridge network can resolve each other by container name via embedded DNS.

Q14. What's the difference between the default bridge network and a user-defined bridge network? The default bridge doesn't provide automatic DNS resolution between containers (requires legacy `--link`), while user-defined bridge networks provide automatic name-based service discovery and better isolation. Best practice is to always create a custom network for multi-container apps.

Q15. How do you limit a container's CPU and memory usage? Using flags like `--memory=512m` and `--cpus=1.5` on `docker run`, or the `deploy.resources.limits` section in a Compose file for Swarm/Compose-managed services.

Q16. What is a multi-stage build, and why use it? It lets a Dockerfile use multiple FROM stages — one for compiling/building, and a final minimal stage that copies over only the compiled artifacts. This keeps the production image small and avoids shipping build tools, source code, or intermediate artifacts.

Q17. How do you pass environment variables into a container? With `-e KEY=value` on `docker run`, an `--env-file`, the `ENV` instruction in a Dockerfile (build-time default), or the `environment:` section in a Compose file.

Q18. What is Docker Compose, and when would you use it over plain Docker commands? Compose is a tool for defining and running multi-container applications from a single YAML file. It's used whenever an app has multiple interdependent services (web server, database, cache, etc.), since it manages networking, startup order, and lifecycle for the whole stack with one command.

Q19. How does layer caching work, and how do you take advantage of it? Docker caches each build layer and reuses it if the instruction and its inputs haven't changed. To maximize cache hits, place instructions that change rarely (installing dependencies) before instructions that change often (copying source code), so a source change doesn't invalidate the dependency-install layer.

Q20. What is the difference between `docker stop` and `docker kill`? `docker stop` sends `SIGTERM` (allowing graceful shutdown) and waits a grace period before

sending SIGKILL. `docker kill` sends SIGKILL (or a specified signal) immediately, force-terminating the container.

Q21. How do you debug a container that keeps crashing/exiting? Check `docker ps -a` for the exit code, view `docker logs <container>` for error output, try running the image interactively (`docker run -it <image> sh`) to inspect the environment, and check `docker inspect` for the configured command/entrypoint and health status.

Q22. What is a health check in Docker, and how do you configure one? HEALTHCHECK in a Dockerfile (or `healthcheck:` in Compose) defines a command Docker periodically runs to determine if a container is “healthy.” This lets orchestration tools detect and restart unhealthy containers automatically.

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost/ || exit 1
```

Q23. What’s the difference between `docker exec` and `docker attach`? `docker exec` starts a new process inside an already-running container (e.g., opening a new shell) without affecting the main process. `docker attach` connects your terminal to the container’s existing main process (PID 1) STDIN/STDOUT/STDERR stream.

Q24. How do you share data between two containers? By mounting the same named volume into both containers, or by using `--volumes-from` to reuse another container’s mounted volumes.

Q25. What is the difference between `docker run` and `docker start`? `docker run` creates a *new* container from an image and starts it. `docker start` starts an *existing* (previously created/stopped) container.

2.3 Advanced Level

Q26. How does Docker differ from Kubernetes? Docker is a containerization platform for building and running individual containers on a single host (with Swarm as its native orchestrator for multiple hosts). Kubernetes is a container **orchestration** system that manages containers (often built by Docker/OCI-compliant tools) at scale across clusters — handling scheduling, self-healing, auto-scaling, service discovery, and rolling updates. Kubernetes doesn’t replace Docker’s image-building role; it replaces Docker’s simpler orchestration (Swarm) for large, complex deployments.

Q27. What is the OCI (Open Container Initiative), and why does it matter? OCI is a standards body defining specifications for container **images** and **runtimes**, so that any OCI-compliant tool (Docker, Podman, containerd, CRI-O) can build, run, and share containers interoperably — you’re not locked into a single vendor’s format.

Q28. Explain how Docker achieves isolation without a hypervisor. Docker uses Linux kernel features directly: **namespaces** (PID, NET, MNT, UTS, IPC, USER) isolate what a container can see, and **cgroups** limit what resources it can consume. Because containers share the host kernel rather than virtualizing hardware, they start faster and use fewer resources than VMs, at the cost of weaker isolation boundaries than a hypervisor provides.

Q29. How would you secure a Docker image and container in production? Use minimal/distroless base images; run as non-root; drop unnecessary Linux capabilities (`--cap-drop`); avoid `--privileged`; scan images for CVEs in CI (Trivy/Snyk/Docker Scout); sign images; never bake secrets into layers; set read-only root filesystems where possible (`--read-only`); apply resource limits; and keep the host and Docker Engine patched.

Q30. What is a “dangling image,” and how do you clean it up? A dangling image is a layer with no tag, usually left behind after rebuilding an image with the same tag (the old layer becomes `<none>:<none>`). Clean up with `docker image prune`, or more aggressively `docker system prune -a` to remove all unused images, containers, networks, and build cache.

Q31. How does Docker handle logging, and how would you manage logs at scale? By default, Docker captures a container’s STDOUT/STDERR using a logging driver (`json-file` by default). For production at scale, you’d configure a driver like `fluentd`, `syslog`, `gelf`, or `awslogs` to ship logs to a centralized system (ELK stack, CloudWatch, Splunk) instead of relying on local `json-file` logs, which can grow unbounded without log rotation settings.

Q32. What is the difference between a bind mount and a named volume in terms of production suitability? Bind mounts tie a container to a specific host filesystem path, which is fragile across different hosts/environments and bypasses some of Docker’s volume management (permissions, drivers, backup tooling). Named volumes are portable, can use pluggable storage drivers (e.g., for cloud block/object storage), and are the recommended approach for production persistent data.

Q33. How would you design a Dockerfile for a compiled language (like Go or Java) to keep the final image minimal? Use a multi-stage build: compile in a full SDK image (`golang`, `maven/gradle` + `JDK`), then copy only the resulting binary/JAR into a minimal runtime image (`alpine`, `distroless`, or a slim JRE image), discarding the entire build toolchain from the final artifact.

Q34. Explain container orchestration and why it’s needed at scale. At scale, manually running/monitoring individual containers doesn’t work — you need automated scheduling across many hosts, self-healing (restarting failed containers), load balancing, rolling updates/rollbacks, secret/config management, and horizontal scaling based on demand. Orchestrators like Kubernetes or Docker Swarm provide these capabilities declaratively, so you describe desired state and the orchestrator reconciles reality to match it.

Q35. What is the purpose of an ENTRYPOINT with a CMD array together, and give an example. `ENTRYPOINT` fixes the executable so the container always behaves like that binary; `CMD` supplies default arguments that a user can still override at docker run time.

```
ENTRYPOINT ["python", "app.py"]  
CMD ["--port", "8080"]
```

Running `docker run myimage --port 9090` overrides just the `CMD` arguments while keeping `python app.py` fixed.

Q36. How does Docker’s overlay network enable multi-host container communication? Overlay networks create a virtual, distributed network spanning multiple Docker hosts (typically in a Swarm cluster), using VXLAN encapsulation so containers on different physical/VM hosts can communicate as if on the same L2 network, with the Docker control plane managing routing and service discovery.

Q37. What is the difference between docker-compose (v1, Python) and docker compose (v2, Go plugin)? `docker-compose` was the original standalone Python tool; `docker compose` (no hyphen) is the newer Go-based implementation built as a Docker CLI plugin, offering better performance, active maintenance, and tighter integration with the Docker CLI. Most modern installs ship the v2 plugin by default.

Q38. How would you handle secrets in a Dockerized application? Avoid `ENV/build-args` for secrets (they leak into image history/layers). Instead use runtime injection via a secrets manager (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault), Docker Swarm/Kubernetes native secrets objects (mounted as files, not env vars, when possible), or `--env-file` supplied only at deploy time and excluded from version control.

Q39. What causes container image bloat, and how do you diagnose it? Common causes: large base images, unnecessary build tools left in the final image, uncleaned package manager caches, unnecessary files copied due to a missing `.dockerignore`, and too many separate RUN layers each leaving residue. Diagnose with `docker history <image>` to see per-layer size, or tools like `dive` to explore layer contents interactively.

Q40. How do rolling updates work with Docker Swarm services? `docker service update` with `--update-parallelism` and `--update-delay` replaces a configurable number of replica tasks at a time with the new image version, waiting between batches, and can automatically roll back if health checks fail — enabling zero-downtime deployments.

2.4 Scenario-Based Questions

Q41. Your container starts fine locally but fails in production with “permission denied” errors. What would you check? Check if the container is running as a non-root USER that lacks permission on a mounted volume/bind-mounted directory; check file ownership on the host path being mounted; check SELinux/AppArmor policies in production that may not exist locally; and verify environment-specific configuration differences (e.g., read-only filesystem flags).

Q42. A container is consuming excessive memory and getting OOM-killed. How do you investigate and fix it? Use `docker stats` to observe live memory usage, check `docker inspect` for any configured `--memory` limit, review application-level memory leaks/logs, and either fix the application’s memory usage or set/adjust `--memory` and `--memory-swap` limits appropriately alongside monitoring/alerting.

Q43. You need zero-downtime deployments for a containerized web app. How would you approach it? Use an orchestrator (Swarm services or Kubernetes Deployments) with rolling update strategy, health checks to gate traffic to only healthy

replicas, and a load balancer/reverse proxy in front so old containers keep serving until new ones pass health checks, then old ones are drained and removed.

Q44. How would you migrate a legacy monolithic app into Docker containers? Start by containerizing the app as-is (lift-and-shift) to validate the environment works in a container, using bind mounts/volumes for any stateful data initially. Then incrementally break out distinct components (database, cache, background workers) into their own containers/services via Compose, refactoring toward smaller services over time rather than a risky big-bang rewrite.

Q45. Two containers on the same host need to communicate, but you don't want to expose ports to the outside world. How do you set this up? Create a user-defined bridge (or overlay, for multi-host) network, attach both containers to it (`--network`), and let them communicate over that internal network using container names for DNS resolution — with no `-p` port publishing needed since only external access requires exposed ports.

2.5 Quick Reference: Most Common Docker Commands

```
# Images
docker build -t name:tag .
docker images
docker rmi <image>
docker pull <image>
docker push <image>

# Containers
docker run -d -p 80:80 --name web nginx
docker ps -a
docker stop/start/restart <container>
docker rm <container>
docker exec -it <container> bash
docker logs -f <container>

# System
docker system df
docker system prune -a
docker inspect <object>

# Compose
docker compose up -d
docker compose down
docker compose logs -f

# Volumes & Networks
docker volume create/ls/rm
docker network create/ls/inspect
```

